

Using AI (LLM) in Manual testing

Set up: set up a proxy for LLM (using BurpSite or OWASP ZAP extension/plugin) or using script so that LLM can access the web application

LLM prompt techniques:

- Chain-of-Thought (CoT) Prompting for Logic Flaws: Force the LLM to break down its reasoning step-by-step. This drastically reduces AI hallucinations. By forcing the model to "show its work," the human tester can easily see where the AI's logic broke down if it generates a bad payload.
- Retrieval-Augmented Generation (RAG) for WSTG Grounding: RAG involves feeding a specific, trusted document database into the LLM's temporary memory. Propose a technique where the tester first loads the target application's Swagger/OpenAPI documentation and the official OWASP WSTG PDF into the LLM's context window.

Input validation testing

- Forcing Structured Outputs (JSON/YAML Integration): Propose standardizing LLM outputs into strict JSON formats.
- Semantic Chunking for Large Client-Side Codebases: A script first breaks the massive JavaScript file down by functions or routing modules, and then feeds those smaller "chunks" to the LLM one by one to map out the API endpoints.

I, Information gathering

- Use Automated tool for spidering / crawling
- Use LLM to scan HTML/ JS source code to detect exposure information
- Use LLM to map out the architecture of web applications
- Use LLM to detect any possible vulnerabilities that can be exploited based on the architecture of the web app

II, Input validation testing

1, XSS testing

- Manual detect I and Use LLM to detect all input attack vector and giving all possible injection and use LLM to write a script to perform that all at one and observe the output
- Validate the input attack vectors that LLM detect and possible injection, validate the result of the script then conclusion

2, HTTP parameter pollution

- Identify the backend and the parsing method used.
- Give LLM the context of backends and business logic and Use LLM to generate script for all possible HPP injection on the back end
- Validate the script , run it and observe the result and giving the conclusion

3, SQL injection testing

- Manually identify the SQL injection point and using LLM to identify the SQL injection and compare the result and validate
- Use LLM to generate the script that could do all possible injection attack on the injection point, validate the script , observe the result of the script and give conclusion

4, LDAP injection

- Manually identify the LDAP injection point and using LLM to identify the LDAP injection and compare the result and validate
- Use LLM to generate the script that could do all possible injection attack on the injection point, validate the script , observe the result of the script and give conclusion

5, XML, SSI , XPATH , IMAP SMTP, Code,... injection

- same above

III, Business logic testing

- Give the LLM the Business logic context

1, Test Business logic data validation

- Manually Identify injection point and use LLM to detect any injection point remaining
- Use LLM to identify how the backend works
- Use LLM to generate a script to test the logic by break the format of expected data. Validate the script generated by LLM and then observe the result of the script and give conclusion

2, Test Ability to Forge Requests

- Give the LLM the Business logic context based on the Project documents
- Use LLM to observe the HTTPS GET/POST request and generate a list of guessable and hidden options. Validate the results of LLM to make sure nothing is forget
- Use LLM to generate list of possible test case that can exploit. Validate the test case , perform and give conclusion to the result

3, Test Integrity Checks

4, Test for Process Timing

- Give LLM the project documentation to identify the system functionality that may be impacted by time
- Use LLM to develop misuse case, validate the result generated by the LLM
- Perform and give conclusion of the result

5, Test Number of Times a Function Can Be Used Limits

- Manually identify the Identify functions that must set limits to the times they can be called.
- Give LLM the context about limited functions
- Use LLM to generate all possible test case, validate the generated test case , perform the test case and observe the results

6, Testing for the Circumvention of Work Flows

- Give LLM the context of workflow and use LLM to analyze identify the possible vulnerabilities, validate the results of LLM
- Use LLM to develop and generate all possible test cases to disrupt the workflow, validate the test cases generated by the LLM

7, Test Defenses Against Application Misuse

8, Test Upload of Unexpected File Types

- Review the project document and give LLM the context of file types that are rejected by system
- Use LLM to generate all possible test cases , validate the test case and perform the test case

9, Test Upload of Malicious Files

- Identify the upload function
- Review the project document and give LLM the context of what file types are considered acceptable, and what types would be considered dangerous or malicious.
- Use LLM to generate all the possible test cases , validated the generated test cases, then perform

Proposed Enhancements: AI-Augmented Testing and the Necessity of Human Verification

1, The Shift to AI-Augmented Manual Testing

The empirical evaluation of the OWASP WSTG methodology demonstrated that while manual testing is highly accurate, it is incredibly time-consuming and difficult to scale. Conversely, automated scanners (like OWASP ZAP or Burp Suite) are fast but lack the contextual awareness needed to identify complex logical flaws. To bridge this gap, this thesis proposes an enhancement to the WSTG framework: Integrating Large Language Models (LLMs) as Augmentative Testing Assistants.

By integrating AI into the testing lifecycle, penetration testers can vastly accelerate repetitive, context-heavy tasks. Proposed implementations include:

Automated Code Parsing and De-obfuscation: Feeding minified client-side JavaScript or complex API documentation into an LLM to instantly map hidden endpoints (WSTG-INFO-05), converting hours of manual reconnaissance into seconds of automated text synthesis.

Semantic Payload Generation: Using AI to analyze a specific business logic workflow (e.g., a checkout cart) and asking the model to generate highly customized, edge-case payloads (like integer overflows or HTTP parameter pollution strings) designed specifically for that architecture.

However, while AI drastically increases testing velocity, relying on LLMs introduces severe, unique risks. Therefore, the WSTG must adopt a strict "Human-in-the-Loop" (HITL) policy. AI must never be granted autonomous execution rights; it must act solely as an advisory tool.

2. The Imperative of the Human Check Step

Integrating AI into security testing is fundamentally different from integrating standard deterministic automation. Because LLMs operate via probabilistic text generation rather than rigid code execution, their outputs are inherently unreliable. A mandatory human verification step must be enforced for everything the LLM generates due to the following critical limitations

2.1 AI Hallucinations and High False-Positive Rates

LLMs are designed to generate plausible-sounding text, not necessarily factual or technically accurate data. In a penetration testing context, an LLM might "hallucinate" a vulnerability by misinterpreting standard application behavior as a critical flaw.

- *Example:* If an LLM analyzes a secure authentication token and incorrectly flags it as a vulnerable MD5 hash, an autonomous system might waste hours trying to exploit a secure system.
- *The Human Check:* A human analyst is required to validate the AI's findings against the actual application architecture to filter out false positives and ensure the testing remains focused on genuine risks.

2.2 Contextual and Semantic Blindness in Business Logic

While AI can parse code syntax, it lacks genuine semantic understanding of real-world business rules. The WSTG emphasizes Business Logic Testing (WSTG-BUS), which dictates that a tester must understand the *intent* of the application.

- *Example:* An AI might review an API endpoint that allows a user to delete a record and flag it as an Insecure Direct Object Reference (IDOR). However, it takes a human tester to understand the business context—specifically, whether the user *should* have the administrative right to delete that record. AI cannot understand corporate hierarchies or authorized workflows; it only understands data input and output.

2.3 Non-Deterministic Payload Generation

Standard automated scanners (like ZAP) use static, peer-reviewed payload lists that produce predictable results. LLMs, however, are non-deterministic; the same prompt will yield different payload suggestions on different days.

Example: If an LLM generates a custom SQL Injection payload, it might accidentally include a DROP TABLE command simply because that phrase frequently appears in its training data regarding SQL injections.

The Human Check: A human tester must manually review, sanitize, and test every AI-generated payload in a sandbox environment. Executing AI-generated exploits blindly risks causing severe data corruption, system crashes, or Denial of Service (DoS) conditions on the target environment.

2.4 Data Privacy, NDAs, and Compliance Risks

Using cloud-based LLMs (such as ChatGPT or Gemini) involves transmitting data to external third-party servers. If a tester blindly feeds application source code, proprietary API schemas, or HTTP responses containing Personally Identifiable Information (PII) into a public AI model, they are actively breaching Non-Disclosure Agreements (NDAs) and data privacy laws (such as GDPR).

- *The Human Check:* The human analyst serves as the critical data-sanitization gateway. Before any application data is sent to an AI for analysis, the human must manually strip out sensitive variables, internal IP addresses, and proprietary tokens, ensuring the testing process remains legally compliant.